

Linux 离线安装包 (flocks-offline.run)

给不能联网的 CentOS Stream 9 / RHEL 9 系机器用的单文件安装包。客户只需要 `scp` 上去、`sudo bash /var/tmp/flocks-offline.run`，装完浏览器开 `http://服务器IP:5173`。包里带齐 Python 3.12、Node.js、uv、预建好的 `.venv`、构建好的 WebUI，可选再带一份 Pro bundle；安装过程和之后的 Pro 升级都不需要 PyPI、npm、GitHub。

这套东西和 `install.sh` / Docker / Windows 包互不影响：装到机器上的形态还是"仓库根 + `.venv` + `webui/dist`"，只是放在 `/opt/flocks/flocks`、以 `flocks` 用户跑、由 `/etc/flocks/flocks.env` 提供环境变量。代码里为它加的逻辑全部由 `FLOCKS_PRO_BUNDLE_DIR`、`FLOCKS_UPDATE_CHANNEL` 和 bundle manifest 的 `prebuilt:true` 触发，这三样只有这个安装包会设置。

内容

- [文件](#)
- [怎么构建](#)
- [装到机器上是什么样](#)
- [Pro 升级在离线机器上怎么走](#)
- [和原来的升级方式是什么关系](#)
- [测试](#)

文件

文件	作用
<code>build-in-docker.sh</code>	一键打包：起 CentOS Stream 9 容器跑下面的构建脚本， <code>--arch x86_64</code>
<code>build-offline-run.sh</code>	真正的构建脚本，在目标发行版的容器里跑（推荐 <code>quay.io/centos/centos:stream9</code> ），产出 <code>flocks-offline.run</code> + <code>.sha256</code> + <code>versions.json</code>
<code>versions.manifest.json</code>	锁定的 Python / Node / uv / makeself 版本与下载地址模板；升级工具链改这里

文件	作用
<code>installer/install.sh</code>	打进包里的安装脚本， <code>sudo bash xxx.run</code> 时由 <code>makeself</code> 自动执行，全程不提问
<code>installer/flocks.env.template</code>	渲染成 <code>/etc/flocks/flocks.env</code>
<code>installer/flocks.service</code>	渲染成 <code>/etc/systemd/system/flocks.service</code>
<code>installer/flocks-cli-wrapper.sh</code>	渲染成 <code>/usr/local/bin/flocks</code> ， <code>sudo flocks status</code> 会以服务用户身份执行
<code>smoke-test.sh</code>	在无网络的一次性容器里装一遍：装、建管理员、本地 Pro 包安装、重启、重复执行升级，输出 PASS/FAIL
<code>verify-install.sh</code>	在装好的真机上跑的验收脚本（root）：布局、systemd、firewalld、建管理员/登录、本地 Pro 包安装、 <code>--rerun</code> 重复执行安装包，输出 PASS/FAIL

CI: `.github/workflows/linux-offline-run.yml` 在 `x86_64 (ubuntu-latest)` 和 `aarch64 (ubuntu-24.04-arm)` 两个原生 runner 上各起一个 CentOS Stream 9 容器构建，手动触发上传两个 artifact（可填 Pro wheel 的 URL 和 release id），发布 Release 时把 `flocks-offline-x86_64.run`、`flocks-offline-aarch64.run` 挂到 Release 资产；交付给客户时改名成说明书写死的 `flocks-offline.run`。

怎么构建

一键（本机有 docker 即可）：

```
packaging/linux/build-in-docker.sh --arch all --flockspro-wheel
  ../flockspro/dist/flockspro-<ver>-py3-none-any.whl
# 产物: packaging/linux/Output/x86_64/flocks-offline.run、
      packaging/linux/Output/aarch64/flocks-offline.run
```

`--arch` 默认是本机架构；另一种架构要 Docker 里有 QEMU（`docker run --privileged --rm tonistiigi/binfmt --install all`），慢好几倍，同架构机器或 CI 矩阵更快。国内机器加 `--cn` 走 `npmmirror`。`--keep-container` 会留下容器，进去 `build-offline-run.sh --repack` 可以只换脚本快速重打。

venv 不能搬家（`pyenv.config`、shebang、`editable .pth` 全是绝对路径），所以构建必须在目标路径 `/opt/flocks` 下进行，要求这个目录为空——这就是一定要在干净容器里跑的原因。手动等价命令：

```
docker run --rm -it -v "$PWD":/src -v ~/.cache/flocks-offline-
    build:/root/.cache/flocks-offline-build \
    quay.io/centos/centos:stream9 bash -c '
    dnf -y install tar gzip findutils which git python3 diffutils procs-ng
    iproute &&
    cd /src && bash packaging/linux/build-offline-run.sh --output-dir
    /root/.cache/flocks-offline-build/out'
```

带 Pro bundle: 加 `--flockspro-wheel /path/flockspro-<ver>-py3-none-any.whl`, 可选 `--pro-release-id` (console 上对应的 release)、`--pro-bundle-version`。wheel 必须是和这个 core 配对的版本。

国内构建机可以走镜像:

```
FLOCKS_NODE_BASE_URL=https://registry.npmmirror.com/-/binary/node、
FLOCKS_NPM_REGISTRY=https://registry.npmmirror.com/、
FLOCKS_UV_DEFAULT_INDEX=https://mirrors.aliyun.com/pypi/simple。下载的工具链缓
存在 ~/.cache/flocks-offline-build, 改 manifest 才会重新下。
```

构建做的事: 下载工具链 → 拷贝仓库 (不含 `.git`、`tests`、`node_modules`) → 用包内 Python 跑 `uv sync --frozen --group dev` → 在 `scratch` 目录 `npm ci && npm run build` 后只把 `dist` 拷进来 → 写 Pro bundle 和 `versions.json` → `makeself` 打包 (`--target /opt/flocks/.staging`, 解包不占 `/tmp`)。x86_64 和 aarch64 都能构建, 产物只能装到同架构机器。

装到机器上是什么样

<code>/opt/flocks/tools/{python,node,uv}</code>	运行时, root 所有
<code>/opt/flocks/flocks</code>	仓库副本 + <code>.venv</code> + <code>webui/dist</code> , flocks 用户所有 (升级要写它)
<code>/opt/flocks/bundle</code>	Pro bundle (<code>manifest.json</code> + <code>wheels/</code>), 有才装
<code>/opt/flocks/cache/uv</code>	构建机的 uv 缓存, <code>UV_OFFLINE=1</code> 时兜底
<code>/opt/flocks/installer, versions.json</code>	这次安装用的脚本和版本信息
<code>/etc/flocks/flocks.env</code>	服务环境变量; 改 IP/端口/控制台地址就改它然后 <code>systemctl restart flocks</code>
<code>/etc/systemd/system/flocks.service</code>	<code>oneshot</code> + <code>RemainAfterExit</code> , <code>ExecStart=flocks start</code> , <code>ExecStop=flocks stop</code>
<code>/var/lib/flocks/.flocks</code>	数据 (配置、数据库、日志、授权), 任何情况下不动
<code>/usr/local/bin/flocks</code>	CLI 包装

安装脚本会：建 `flocks` 用户；已有安装先停服、把旧的 `flocks/`、`tools/`、`bundle/` 改名成 `*.bak-<时间戳>`（不删）；搬入新文件后 `restorecon -R`（从临时目录 `mv` 过来的文件会带 `tmp_t` 标签，SELinux enforcing 下 `systemd` 起不来）；合并 `flocks.env`（受管的几个键覆盖，其它保留）；如果之前激活过 Pro（`~/.flocks/run/pro-bundle-installed.json` 在），把包里的 Pro wheel 重新装进新 `venv`；`systemctl enable --now`；`firewalld` 放行 `5173/tcp`；`chronyd` 打开；轮询 `/api/health` 直到 200；最后打印 `[flocks]` 安装完成：`http://<IP>:5173`。日志在 `/var/log/flocks-offline-install.log`。

`systemd` 用 `oneshot` 而不是把 `service-daemon` 放前台，是因为在线升级的 `restart_handoff` 会先 `stop_all` 再自己拉起新的 `flocks start`；前台模式下主进程一退 `systemd` 会连 `handoff` 一起杀掉。`oneshot` 下 `systemctl status` 只能看到 `active (exited)`，看进程用 `sudo flocks status`。

调试用的环境变量（客户不需要）：`FLOCKS_OFFLINE_DRY_RUN=1` 只打印计划；`FLOCKS_OFFLINE_SKIP_SYSTEMD=1` 在容器里用 `flocks start` 直接起；`FLOCKS_OFFLINE_SKIP_FIREWALL=1`、`FLOCKS_OFFLINE_SKIP_START=1`、`FLOCKS_OFFLINE_PORT`、`FLOCKS_OFFLINE_HEALTH_TIMEOUT`。

Pro 升级在离线机器上怎么走

页面上"开始升级"调的还是 `perform_pro_bundle_install`。它先看 `FLOCKS_PRO_BUNDLE_DIR`（安装包写进 `env` 的 `/opt/flocks/bundle`）：目录里有 `manifest.json` 就用本地包，不下载；`manifest` 标了 `prebuilt: true` 且没带 `flocks/` 源码，就只做 `uv pip install --no-deps <wheel>`，跳过 `uv sync` 和 `npm`，写 `marker` 后走 `handoff` 的 Pro-only 重启。`console` 的 `manifest` 只是尽力拉一次，版本相同就把 `release_id` 合进 `marker`，拉不到或版本不同都不影响安装。

`FLOCKS_UPDATE_CHANNEL` 被安装包固定成 `flockspro-offline-<core 版本>`，心跳和升级申请都会带上这个 `channel` 字段；`console` 给每个离线包版本开一个冻结渠道、发和包里同一个 `release`，在线客户的 `flockspro` 渠道不受影响。老客户端不带 `channel` 字段，`console` 按 `flockspro` 处理。

`prebuilt` 格式的 `bundle` 只能发到离线渠道。老版本客户端拿到没有 `flocks/` 目录的包会判成"不是 Pro bundle"，直接报"未找到 `flockspro wheel`"。

和原来的升级方式是什么关系

离线实例的升级机制和在线安装是同一套代码（备份到 `~/.flocks/version`、`_replace_install_dir` 保留 `.venv`、`marker`、`handoff` 重启），只是"包从哪来"不同：Pro 组件来自包内 `bundle/`，核心升级靠新版 `.run` 覆盖执行。`env` 里的

FLOCKS_DEPLOY_MODE=offline 会让页面"检查更新 → 升级"、flocks update 和 /api/update/apply 在动任何文件之前就拒绝联网升级并提示用新 .run （否则会先替换源码再在 uv sync 上被 UV_OFFLINE=1 卡住，服务停在半路）。页面上 update_allowed=false 时显示"离线安装包部署"的提示，不是 Docker 那条。

机器如果后来能联网、想改回在线升级：从 /etc/flocks/flocks.env 删掉 FLOCKS_DEPLOY_MODE / FLOCKS_OFFLINE_INSTALL / UV_OFFLINE / npm_config_offline / FLOCKS_PRO_BUNDLE_DIR / FLOCKS_UPDATE_CHANNEL，systemctl restart flocks，之后就是普通源码安装的在线升级路径（GitHub/Gitee 源码包 + console flockspro 渠道），代码不用改。反过来，install.sh 装的实例不能用 .run 原地升级：.run 是另一种部署形态（/opt/flocks、flocks 用户、/var/lib/flocks/.flocks），数据要自己从 ~/.flocks 搬过去，而且 5173 被旧实例占着时安装脚本会直接报错。

在线用户不受影响：新逻辑全部靠 FLOCKS_PRO_BUNDLE_DIR、FLOCKS_DEPLOY_MODE=offline、manifest 的 prebuilt 和 handoff 的 --prebuilt 参数触发，console 现在发的 bundle（带 flocks/）新代码照收，既有 updater 测试全部原样通过。

测试

```
uv run pytest tests/scripts/test_linux_offline_installer.py # 语法、无递归删除、模板键、dry-run
uv run pytest tests/updater/test_updater_prebuilt_bundle.py # 本地 bundle / prebuilt 路径，以及默认路径不变的回归
packaging/linux/smoke-test.sh Output/flocks-offline.run # 无网络容器里真装一遍（需要 docker，镜像架构要和包一致）
sudo packaging/linux/verify-install.sh --port 5173 --rerun /var/tmp/flocks-offline.run # 在装好的真机上验收
```

真机上 5173 已被别的服务占用时，安装和验收都可以换端口：FLOCKS_OFFLINE_PORT=5273 sudo bash /var/tmp/flocks-offline.run、verify-install.sh --port 5273。安装脚本发现端口被占且没有旧的 Flocks 安装会直接报错退出，不会去停别人的进程。

真机验收清单在 webook_dev/docs/offline_install/ 的差距分析文档里（断外网只放行 portal/passport 的 CentOS Stream 9 x86_64，按客户说明书原文从 scp 跑到 Pro 激活、两次重启、重复执行 .run）。